Arquitectura en Capas + ORM

Clase 03 — Desarrollo de Software 2026

Hoy no escribimos una API nueva. **La reorganizamos para que crezca sin caerse.**

Objetivos de la clase

Al terminar, vas a poder:

1. **Separar** una API en 3 capas — Controller → Service → Repository — y explicar **por qué** cada cosa vive donde vive.
2. **Usar un ORM** (SQLModel) para hablar con la base en lugar de escribir SQL crudo como en el Módulo 02.
3. **Tipar el frontend** con TypeScript, reflejando el contrato de la API como tipos.

> *Tres objetivos, 60 minutos de actividad, y los construís vos en grupo.*

Quiz de apertura (aula invertida)

1. ¿Cuáles son las **3 capas** y qué hace cada una?
2. ¿Por qué el **service** no debería devolver un `404`?
3. ¿Qué es un **ORM** y qué diferencia hay con el SQL del Módulo 02?

> *El quiz sale del `MATERIAL_PREVIO.md` (con su bibliografía). Si no lo leíste, hoy vas a ver pasar la clase por la ventana.*

El problema: el monolito

El `main.py` del Módulo 02, **casi 300 líneas**, mezclaba todo:

```
# modelos Pydantic
class TaskCreate(BaseModel): ...
class Task(BaseModel): ...
# pool de conexiones
pool = ConnectionPool(...)
# SQL crudo DENTRO de los endpoints
@app.post("/api/tasks")
def create_task(body: TaskCreate):
    with pool.connection() as conn:
        conn.row_factory = dict_row
        with conn.cursor() as cur:
            cur.execute("INSERT INTO tasks ... RETURNING ...", (...))
```

> *Cambiar UNA cosa obliga a tocar código que no tiene nada que ver.* ***Eso es un monolito.***

La solución: capas con regla de dependencia

```
Controller ──▶ Service ──▶ Repository ──▶ Base de datos
  (HTTP)        (negocio)     (ORM/SQL)
```

Capa	Oficio	NO sabe de…
Controller	rutas, status codes, JSON	SQL, reglas de negocio
Service	reglas de negocio	HTTP, SQL
Repository	SQL/ORM, la base	HTTP, negocio

Regla de oro: una capa solo conoce a la de **abajo**. Nunca hacia arriba.

> *El `404` es HTTP → vive en el **controller**. El service devuelve `None` y **no sabe qué es un status code**. Esa es LA lección de hoy.*

El mapa de las capas (dónde vive cada cosa)

```
backend/app/
├── main.py              → entrypoint (app + routers + lifespan)
├── database.py          → conexión (engine + session + create_all)
├── dependencies.py      → cableado (Depends: Session → Repo → Service)
│
├── models/task.py       → SQLModel (la tabla + schemas)
├── repositories/        → ORM (select, get, add, update, delete)
├── services/            → reglas de negocio (strip, "no existe")
└── controllers/         → endpoints HTTP (el 404 vive acá)
```

> *Cada carpeta = una capa = **una sola responsabilidad**. Ese es el edificio que vas a construir hoy.*

Y encima: el ORM

Módulo 02 — vos escribías el SQL:

```
cur.execute("SELECT id, title, completed, created_at FROM tasks ORDER BY id")
```

Módulo 03 — el ORM lo escribe por vos:

```
statement = select(Task).order_by(Task.id)
tasks = session.exec(statement).all()
```

> *Pensás en **objetos** (`Task`), no en filas y columnas. El ORM traduce.*
>
> *Y `created_at` ya no se serializa a mano: SQLModel + Pydantic lo resuelven.*

Dónde vive cada cosa (antes → después)

Responsabilidad	Módulo 02 (monolito)	Módulo 03 (capas)
Validación (Pydantic)	en el endpoint	`models/task.py`
SQL / acceso a datos	dentro de los endpoints	`repositories/`
Regla de negocio (`strip` , "no existe")	dentro de los endpoints	`services/`
HTTP (rutas, 404, status codes)	mezclado	`controllers/`
Conexión a la base	`pool` arriba del main	`database.py`
Cableado entre capas	implícito	`dependencies.py`

> *El `strip()` es **negocio** → service. El `404` es **HTTP** → controller.*
> *El `SELECT` es **datos** → repository. Cada cosa, en su casa.*

El plan — 60 min de actividad

> *La lectura previa ya la hiciste en casa. Acá van **60 minutos de construcción**.*

Min	Fase	Qué construyen
0-5	**Setup**	Levantar el scaffold
5-20	**Repository**	Los 6 métodos del CRUD (ORM)
20-30	**Service**	Lógica de negocio (el `None`)
30-40	**Controller**	Endpoints (el `404` acá)
40-50	**Frontend TS**	Conectar y verificar el CRUD
50-60	**Cierre**	Reflexión + descubrimientos

> ***Regla del taller**: el docente NO da respuestas. Hace preguntas.*
> *"¿Eso es HTTP o es negocio?" es la pregunta que lo destraba todo.*

El flujo: fork → clonar → desarrollar

1. **Fork** — en GitHub, creás tu copia del repo.
2. **Clonar** — bajás TU fork a tu máquina.
3. **Desarrollar** — completás las 3 capas siguiendo la guía.
4. **Verificar** — Postman + frontend, todo contra TU backend.

```
git clone https://github.com/TU_USUARIO/backend.git
cd backend/03-arquitectura-en-capas
```

> *Cada uno trabaja en **su fork**. La solución se libera al final de la clase (rama `solucion`) para que compares tu código.*

Fase 0 — Setup del scaffold (5 min)

```
cd 03-arquitectura-en-capas/backend
cp .env.example .env          # DATABASE_URL del Módulo 02
uv sync                       # instala SQLModel + FastAPI
uv run -m app.main            # server en :8000
```

```
curl http://localhost:8000/api/health
```

```
{ "status": "Funciona", "service": "03-...",
  "db": "conectada", "tasks_count": 0 }
```

> *El health **ya funciona** — es el ejemplo vivo que les dejamos en el scaffold.*
>
> *La tabla se crea sola (`create_all`). Ya no hay `schema.sql`.*

✅ Checkpoint 1 — Scaffold arriba

- ☐ `uv sync` sin errores
- ☐ `uv run -m app.main` corriendo
- ☐ `/api/health` responde `db: "conectada"`
- ☐ `/docs` muestra la API

Fase 1 — Repository: leer (15 min)

Completá `app/repositories/task_repository.py`:

```python
class TaskRepository:
    def __init__(self, session: Session):
        self.session = session
    def list_all(self) -> list[Task]:
        ...                     # select(Task) + order_by
    def get_by_id(self, task_id: int) -> Task | None:
        ...                     # session.get(Task, id)
    def count(self) -> int:
        ...                     # ejemplo resuelto
```

> *`select(Task)` reemplaza al `SELECT * FROM tasks` del Módulo 02.*
>
> *El `session.get()` reemplaza al `WHERE id = %s`.*

Fase 1 — Repository: escribir

```
    def create(self, title: str) -> Task:
        ...                     # add + commit + refresh

    def update(self, task, data) -> Task:
        ...                     # model_dump(exclude_unset=True)

    def delete(self, task: Task) -> None:
        ...                     # delete + commit
```

> `exclude_unset=True` es la clave del PATCH: **solo** pisa los campos que vinieron en el body. Sin eso, borrás datos sin querer.

Fase 2 — Service (10 min)

Completá `app/services/task_service.py`:

```
class TaskService:
    def __init__(self, repository: TaskRepository):
        self.repository = repository
    def create_task(self, body: TaskCreate) -> Task:
        return self.repository.create(body.title.strip())   # negocio
    def update_task(self, task_id, body) -> Task | None:
        task = self.repository.get_by_id(task_id)
        if task is None:
            return None                  # ⬅️ NO lanza 404 acá
        return self.repository.update(task, body)
```

> ***La pregunta del millón**: ¿el service devuelve `None` o lanza un `404`?*
>
> *`404` es HTTP, y el service **no sabe qué es HTTP**. Devuelve `None`.*

Fase 3 — Controller (10 min)

Completá `app/controllers/task_controller.py`:

```python
@router.get("/{task_id}", response_model=TaskRead)
def get_task(task_id: int, service: TaskService = Depends(get_task_service)):
    task = service.get_task(task_id)
    if task is None:
        raise HTTPException(404, detail=f"Tarea {task_id} no existe")
    return task
```

Endpoint	Método	Nota
`/api/tasks`	GET	`response_model=list[TaskRead]`
`/api/tasks`	POST	`status_code=201`
`/api/tasks/{id}`	GET / PATCH / DELETE	`None → 404`

> *El controller es el único que "habla HTTP": traduce `None → 404`. Cero SQL acá.*

✅ Checkpoint 2 — CRUD completo por capas

```
curl -X POST http://localhost:8000/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"  Probar capas  "}'   # → "Probar capas" (¡strip!)

curl http://localhost:8000/api/tasks
curl http://localhost:8000/api/tasks/1
curl -X PATCH http://localhost:8000/api/tasks/1 \
  -H "Content-Type: application/json" -d '{"completed":true}'
curl -X PATCH http://localhost:8000/api/tasks/1 \
  -H "Content-Type: application/json" -d '{"title":"Editado"}'
curl http://localhost:8000/api/tasks/999    # → 404
curl -X DELETE http://localhost:8000/api/tasks/1
```

- ☐ El `strip()` normaliza el título (regla de negocio en el service)
- ☐ El `404` funciona (traducción del controller)

Fase 4 — Frontend TypeScript (10 min)

Ya viene **completo**. Levantalo y hacé el CRUD desde la UI:

```
cd ../frontend
pnpm install && pnpm dev        # :5173
```

```
// src/types.ts — el contrato de la API, como TIPO
export interface Task {
  id: number;
  title: string;
  completed: boolean;
  created_at: string;
}
```

> *Este `interface` es el `TaskRead` del backend, pero en TypeScript.*
>
> *El compilador te avisa si escribís mal un campo. Antes, JS callaba.*

✅ Checkpoint 3 — Frontend integrado

- ☐ CRUD completo desde la UI (crear, editar, toggle, eliminar)
- ☐ `types.ts` refleja el contrato del backend
- ☐ Errores de tipo los atrapa el compilador, no el navegador

🔧 Errores comunes (desbloqueo rápido)

Síntoma	Pista para destrabar
`ModuleNotFoundError: app`	¿Estás en `backend/` ? Ejecutá `uv run -m app.main`
`relation "tasks" does not exist`	Revisá `DATABASE_URL` en `.env`
No sabés leer una fila por id	`session.get(Task, id)`
El update pisa campos que no vinieron	`model_dump(exclude_unset=True)`
No sabés dónde va el `404`	*"¿El service sabe qué es un status code?"*
Frontend no conecta	El proxy ya está en `vite.config.ts` (scaffold)

> ***Regla**: no des la respuesta. Hacé la pregunta que la destraba.*
> *"¿Eso es HTTP o es negocio?" resuelve la mitad de las dudas.*

Esto que construyeron tiene nombre

Layered Architecture (Fowler, *PoEAA*) — paquete **por capa técnica**.
Es lo que acabás de armar:

```
app/  controllers/  services/  repositories/  models/
```

El **otro eje** posible: paquete **por feature** (partición vertical):

```
app/  tasks/{controller,service,repository}.py
      projects/{...}
      shared/     (db, config — el kernel)
```

Pregunta	Por capa	Por feature
Agregar `projects`	tocás 4 carpetas	tocás 1 carpeta
"¿Qué hace el sistema?"	no se ve	grita: tasks, projects
Reusar entre features	fácil	cuidado: `shared/` se vuelve basurero

> ***Screaming Architecture**: el árbol grita el **dominio**, no el framework.*
> ***Hexagonal** es el escalón siguiente: puertos (el service depende de una abstracción, no de una clase concreta).*
> ***No es dogma**: dominio chico → simplificá. Dominio que crece y varios equipos → el eje vertical paga solo.*

Hoy no escribieron una API nueva

La reorganizaron para que el día de mañana, cuando crezca, no se caiga

> *"La Universidad te da el mapa. El recorrido lo hacés vos."*